B.Sc. (Business Analytics) Dissertation

# Pairwise Consistency Matching (PCM) Watermark –

# A Semantic-Preserving Extension of the Kirchenbauer's Greenlist Framework

By

Phung Nguyen Hanh

Department of Information Systems

School of Computing

National University of Singapore

2025/2026

B.Sc. (Business Analytics) Dissertation

# Pairwise Consistency Matching (PCM) Watermark –

# A Semantic-Preserving Extension of the Kirchenbauer's Greenlist Framework

By

Phung Nguyen Hanh

Department of Information Systems

School of Computing

National University of Singapore

2025/2026

Project No: H0041770

Advisor: Assoc. Prof. Chang Ee-Chien

Deliverables:

Report: 1 Volume

**Abstract**

The proliferation of large language models has increased the need for reliable detection of machine-generated text, especially with post-generation editing. In this work, we present Pairwise Consistency Matching (PCM), a zero-bit watermarking method that encodes keyed local relational constraints within token blocks rather than global token-frequency bias. PCM detects watermarks by combining anchor-based consistency signal with phase-adjusted block voting. We evaluate PCM against Kirchenbauer-style Baseline and Yoo et al.'s Multi-bit Watermark via Position Allocation under simulated paraphrase, shuffle, and deletion attacks at 400-token and 600-token generation lengths. Across settings, all methods remain highly detectable on clean text. Under perturbations, despite limited robustness, PCM's performance attains healthy margins above chance and improves with generation lengths, suggesting a viable foundation for robustness-oriented refinement. In addition, PCM demonstrates strong practical advantages in runtime efficiency and text quality. These findings position PCM as a promising semantic-preserving greenlist extension, while motivating future work on edit-invariant robustness, adaptive threat models, and broader model and domain validation. Code is released here: https://github.com/pnghaanh/BT4101-Dissertation.

**Subject Descriptors:**

E.3: Data Encryption

G.3: Probability and Statistics

I.2.7: Natural Language Processing

K.6.5: Security and Protection

**Keywords:**

Large language models, zero-bit watermark, provenance detection, semantic-preservation, adversarial perturbations, Pairwise Consistency Matching

**Implementation Software and Hardware:**

Linux (x86_64), Python 3.10.12, Conda, PyTorch, GPT-2

**Acknowledgement**

I would like to express my sincere gratitude to my supervisor Assoc. Prof. Chang Ee-Chien and my advisor Graduate Student Qiu Yupeng for their invaluable guidance and support throughout this project from the very start to end.

# List of Figures

Figure 1. Attack-wise robustness at 400 tokens (Baseline vs. PCM) 15

Figure 2. Attack-wise robustness at 600 tokens (Baseline vs. PCM) 16

Figure 3. Attack-wise robustness at 400 tokens (Multi-bit vs. PCM) 17

Figure 4. Attack-wise robustness at 600 tokens (Multi-bit vs. PCM) 18

Figure 5. Length sensitivity of attacked overall AUC 19

Figure 6. Quality, calibration, and efficiency summary (400 tokens) 22

Figure 7. Quality, calibration, and efficiency summary (600 tokens) 22

# Table of Contents

**Title Abstract**.....i ii
**Acknowledgement**.....iii
**List of Figures**.....iv
**1. Introduction**.....1
**2. Literature Review**.....2
**3. Related Works**.....4
3.1 Baseline (Zero-bit) Watermark.....4
3.2 Multi-bit Watermark.....4
3.3. Motivation for PCM.....4
**4. PCM Watermark**.....5
4.1 High-level Design Objective.....5
4.2 Generation.....5
4.3. Detection.....9
**5. Experimental Setup**.....12
5.1 Dataset, Model Selection, and Hyperparameters.....12
5.2 Attack Strategies.....13
5.3 Evaluation Metrics.....14
**6. Evaluation**.....14
6.1 Robustness.....14
6.2. Text Quality.....20
6.3 Summary of Findings.....22
**7. Discussion**.....23
**8. Limitations**.....28
**9. Conclusion**.....29
**Appendix**.....i
A.1 Full Algorithms.....i
A.2 Hyperparameters and Evaluation Metrics.....v
A.3 Extended Results.....vii
**References**.....x

# 1. Introduction

The widespread integration of large language models (LLMs) into writing tools, educations, and content creation has necessitated the fundamental question of how to distinguish machine-generated text from human-written content. As these systems evolve towards human-like fluency, traditional detection methods based on stylistic anomalies become less effective. This poses challenges regarding integrity enforcement and regulatory compliance, ensuring rightful authorship attribution, as well as combating misinformation.

Watermarking has emerged as a practical way to track provenance without bulky external metadata or major usability loss. It embeds subtle perturbations into token-selection probabilities, which are statistically verifiable by authorised detectors while imperceptible to human eyes. An effective scheme should satisfy three criteria: **stealth** – minimal exploitable artefacts while preserving quality and semantic diversity, **robustness** – resistance to post-hoc edits, and **detectability** – confident signal recognition on clean text. Despite substantial progress, current methods still struggle to achieve all three simultaneously, often trading robustness against stealth or requiring long texts for reliable detection.

In this work, we introduce Pairwise Consistency Matching (PCM), a structured zero-bit watermark that operates through local relational constraints. Beyond increasing the marginal rate of greenlist tokens, PCM enforces keyed colour-consistency rules between anchor and non-anchor tokens within fixed post-warmup blocks, with greenlists computed deterministically from prefix-seeded hashing. Detection combines phase-aware block voting and global consistency scoring into a single decision statistic. We evaluate PCM against Kirchenbauer's Baseline (Zero-bit) and Multi-bit Watermarking via Position Allocation under paraphrase, shuffle, and deletion attacks, and show that PCM provides promising robustness while promoting generation efficiency and text quality.

**Statement of Contribution:** This dissertation makes three contributions. First, it introduces **PCM as a relational zero-bit watermarking design** that shifts evidence from marginal token-colour counts to anchor-conditioned consistency patterns. Second, it provides a **controlled empirical comparison** showing how this relational design changes the robustness-efficiency-text quality trade-off under common post-generation perturbations. Third, it offers **implementation-level evidence** that relational watermarking can remain computationally lightweight and semantic preserving in end-to-end generation pipelines.

We choose zero-bit watermarking deliberately to prioritise detection reliability and deployment practicality over payload transmission. While multi-bit methods provide message capacity, they also impose tighter positional constraints that can be fragile under editing. PCM follows this zero-bit objective, but our findings indicate that structural relational designs still require further work to achieve robust post-hoc detectability. This motivates future semantic-preserving, edit-invariant watermark variants that retain PCM's efficiency while improving resilience.

# 2. Literature Review

Early LLM watermark implementations are strongly influenced by Kirchenbauer et al.'s framework (2023), which splits the vocabulary into green- and red-lists. Token probabilities for those belong to the greenlist are gently boosted to create a statistically detectable signature. The pattern of favoured tokens should be imperceptible to humans but detectable via hypothesis testing on token frequency distribution, which determines whether green tokens appear significantly more than expected in a non-watermarked text.

In 2024, Christ et al. formalises watermarking schemes that only holders of secret key can spot to maximise stealth. Subsequent analysis by Kirchenbauer et al. (2024) discovered that even after paraphrasing by humans or non-watermarked LLMs, sufficient leaked n-gram fragments allow detection at high confidence with approximately 800 observed tokens, which is demanding. Shorter corpa remain highly vulnerable to augmentation attacks, especially if an attacker can recognise tokens that appear more often than others. These limitations are key motivations to improve detection efficiency and strength.

Beyond one-bit red/greenlists watermarks, several works explore Multi-bit watermarks, where messages are embedded in the generated output as binary 0/1 token strings, with extensions incorporating variable embedded messages, stronger hash functions, and entropy-based detection. Some notable works include Jiang et al. (2025)'s StealthInk and Yoo et al. (2024)'s Multi-bit Watermark via Position Allocation. While these methods are harder for attackers to reverse-engineer since they perform encodings at a higher level, capacity constraint arises as synonym availability varies by topic, and detection by embedding-space analysis can be computationally expensive.

Recent works explore ways to make watermarks harder for attackers to detect and remove. In addition to Ren et al. (2024)'s SemaMark, Google DeepMind's SynthID (2023) propose diffused watermarks where the signal is spread across the model's entire probability distribution. Instead of boosting a specific greenlist, subtle, continuous shifts are applied to logits in many directions. While stealth improves significantly, robustness lags behind as paraphrasing removes most detectable structure. Compared to red/greenlist schemes, this method possesses weaker statistical separability and require extensive data for detection.

Despite rapid progress, significant research gaps remain in LLM watermarking. Current methods rely on either statistical token biases that become visible to dedicated attackers, or fine-tune models that limit deployment on lightweight, closed-source systems. Jovanović et al. (2024) demonstrates how querying a watermarked model can be used to reverse its parameters and steal the watermark, enabling both spoofing and scrubbing of watermark signals; while Chen et al. (2024) introduce De-mark, which can systematically remove n-gram watermarks using strategic queries. This demonstrates how watermarks can be reverse engineered, spoofed, or scrubbed entirely throughout the entire model queries cheaply. Many approach struggle with watermark detection in short texts, or texts with low entropy, as reliable signals emerge only after hundreds of tokens. Stealth-oriented methods reduce detectability but often sacrifice robustness, while robust multi-bit schemes require high synonym availability and computationally costly embedding-space analysis.

It is noteworthy that amongst available watermarking methods, Robust Multi-bit Text Watermark with LLM-based Paraphrasers proposed by Xu et al. (2024) is the only one that can survive aggressive word substitution and complete paraphrasing. However, the method requires fine-tuned models that are not easily applied to closed-source LLMs, thus beyond the scope of our research.

These limitations reveal that the field lacks a watermarking approach that is robust to adversaries, stealthy under scrutiny, efficient in texts of various lengths, semantic-quality preserving, and deployable in closed-model settings without heavy finetuning.

# 3. Related Works

## 3.1 Baseline (Zero-bit) Watermark

Baseline watermarking follows the context-seeded greenlist scheme of Kirchenbauer et al. (2023). At each decoding step, a keyed pseudorandom function maps the recent prefix to a green/red partition of the vocabulary, and sampling is nudged toward green tokens through positive logit bias. Detection replays the same keyed partitioning, counts greenlist memberships over generated positions, and applies a binomial-style z-test against the null expectation. The watermark is flagged when the statistic exceeds a fixed threshold.

This design is appealing for three reasons: simplicity, low computational overhead, and strong detectability. However, in practice, modifying one token can have downstream effects on 2-3 tokens (Kirchenbauer et al., 2023), weakening the final test statistic.

## 3.2 Multi-bit Watermark

Multi-bit watermarking via Position Allocation (Yoo et al., 2024) extends keyed partitioning to payload encoding. Instead of a binary green/red split, each step partitions the vocabulary into b keyed color groups. The encoder biases sampling toward the group associated with the current message symbol, producing a position-dependent symbol pattern over the sequence. Detection reconstructs these keyed groups, aggregates count by symbol position, decodes the maximum likelihood symbol sequence, and evaluates significance.

Its main advantage is functionality: it supports message-bearing watermarking rather than pure zero-bit detection. The cost is increased sensitivity to perturbation. With a short one-token local context prefix, symbol evidence is highly position-dependent, so insertions, deletions, and paraphrases can desynchronise trajectories and weaken recoverability.

## 3.3. Motivation for PCM

Baseline and Multi-bit watermarking define two useful design bases: Baseline is simple and detectable but can over-constrain token choice, while Multi-bit introduces structure for encoding but can be sensitive to perturbations and decoding instability. PCM hopes to emerge as a synthesis that targets not only robustness, but also text quality and runtime efficiency.

Conceptually, PCM retains keyed pseudorandom partitioning for reproducibility and adversarial uncertainty but reframes the watermark signal as in-block relational consistency between anchor and non-anchor positions rather than marginal token-frequency effects. This intends to make detection less dependent on a single global count while reducing generation-side interference as only non-anchor positions are biased.

The design rationale is therefore threefold: improve post-edit signal persistence, preserve lexical naturalness by limiting per-token forcing, and reduce decoding overhead through sparse logit modification. Therefore, PCM is not only a robustness mechanism, but a practical efficiency- and quality-oriented extension of zero-bit watermarking.

# 4. PCM Watermark

## 4.1 High-level Design Objective

Pairwise Consistency Matching (PCM) is a structured zero-bit watermark that embeds signal through colour-consistency relations. After warmup period, PCM organises generation into eight-token blocks. For each position, a secret deterministic pattern bit specifies whether the token should follow the same or opposite colour as the block anchor. Here, colour is defined by membership in a prefix-seeded greenlist computed using a secret-keyed hash. The generator then modifies logits toward the target colour to enforce this relation.

The PCM watermark has three defining properties. First, **prefix-seeded randomness** where the greenlist is always deterministically recomputed from the prefix via a secret-keyed hash. Second, **block-structured consistency pattern** where watermark signal is carried by same/flip consistency relative to the anchor across post-warmup block positions. Third, **detection through global consistency statistic and phase-aware block voting** where the detector computes token-level consistency indicators, searches candidate block phases, scores unusually consistent blocks, and combines block-level evidence with global consistency evidence into a final z-score decision statistic.

PCM key methodological shift is to encode a relational constraint i.e. same/opposite colour relative to anchor colour rather than absolute colour targets per position. This produces a detection statistic based on consistency dynamics. Signal resides in pairwise structured dependency rather than a simple token-membership watermark.

## 4.2 Generation

### 4.2.1 Notation

Let $V$ denote the vocabulary space. Let the generated token sequence be

$$x_{0:T-1} = (x_0, x_1, \ldots, x_{T-1}) \text{ where each } x_i \in V \text{ and } |V| = N.$$

The following notations are used:

| **Symbol** | **Meaning** |
|---|---|
| $K$ | Secret watermark key |
| $N$ | Vocabulary size |
| $\gamma \in (0, 1)$ | Greenlist ratio |
| $g = \lfloor \gamma N \rfloor$ | Greenlist size |
| $w$ | Warm-up length |
| $B$ | Block size |
| $i$ | Current token position |
| $m_i = i - w$ | Watermark-time index |
| $b_i = m_i \% B$ | Position within block |
| $a(i) = i - b_i$ | Anchor index i.e. index of first token of current block |
| $p_i$ | Prefix used for greenlist seeding at step $i$ |
| $G_i \subseteq V$ | Greenlist at step $i$ |
| $r_b \in \{0, 1\}$ | Secret deterministic pattern bit for block offset $b$ |
| $z_i(v)$ | Base logit for vocabulary item $v$ at position $i$ |
| $c_i = 1[x_i \in G_i]$ | Colour of token $x_i$ under $G_i$ |

*Table 1.* Notations used in PCM watermarking

### 4.2.2 Prefix-seeded greenlist Construction

PCM relies on the same reproducibility principle seen in both Baseline and Multi-bit watermarks based on seeded greenlists. For each generation position $i$, the greenlist is derived from a short local prefix of length up to 4 i.e.

$$p_i = (x_{max(0,\, i-4)}, \ldots x_{i-1})$$

Compute a seed from the secret key and prefix $s_i = H(K \,||\, p_i)$ where $H$ is a cryptographic hash function such as SHA-256. Using $s_i$, sample a keyed permutation $\pi_i$ over vocabulary indices. The greenlist is the first $g$ elements:

$$G_i = \{\pi_i(1), \pi_i(2), \ldots, \pi_i(g)\}$$

This construction ensures that for every party that has access to $K$, the greenlist is deterministic, while for parties that do not know $K$, greenlist assignments are unpredictable.

### 4.2.3 Warmup, Block Indexing, and Pattern Bits

Watermark begins only after the warm-up region to avoid imposing watermark when the model is still strongly constrained by the prompt, and to create a stable post-warmup region where block-level structure can be imposed. For each position $i \geq w$,

$$m_i = i - w, \qquad\qquad b_i = m_i \,\%\, B.$$

Tokens are then partitioned into consecutive blocks of length $B$. The first token $a_i$ of each block serves as the anchor. Each block is encoded with a secret deterministic binary pattern $r = (r_0, r_1, \ldots, r_{B-1})$, $r_b \in \{0, 1\}$ derived from the secret key. The bit $r_b$ specifies the desired relation between the colour of the current token and the colour of the anchor:

$$r_b = \begin{cases} 1, \text{ same colour relation,} \\ 0, \text{ opposite colour relation} \end{cases}$$

In other words, when $b_i = 0$, $x_i$ is the anchor token is generated normally. Else, when $b_i > 0$, $x_i$ is generated based on whether it shares the same colour as the anchor token $x_{a(i)}$.

### 4.2.4 Anchor-Based Local Dependency in Generation

For each controlled position $i \geq w$, $b_i > 0$, the generator first reconstructs $G_i$ from the last four tokens, then evaluates the colour of the anchor token under that same greenlist:

$$c_{a(i)}^{(i)} = 1[x_{a(i)} \in G_i]$$

Given pattern bit $r_b$, define target colour for position $i$:

$$y_i = \begin{cases} c_{a(i)}^{(i)}, & r_b = 1, \\ 1 - c_{a(i)}^{(i)}, & r_b = 0 \end{cases}$$

Each non-anchor token in a block is not controlled independently. Instead, it is linked to the first token of the same block through a same/opposite colour rule based on the secret pattern bit. This forms the core local dependency structure of PCM.

### 4.2.5 Logit Modification

Let $z_i(x)$ denote the model logit for vocabulary item $v$ at position $i$. Let $G_i$ be the greenlist and $y_i \in \{0, 1\}$ be the target colour defined above. Construct modified logits $z'_i(v)$:

If target is green, double the green-token probability mass:

$$z'_i(v) = \begin{cases} z_i(v) + ln2, & v \in G_i, \\ z_i(v), & v \notin G_i \end{cases}$$

Else if target is non-green ($y_i = 0$), green-token probability is suppressed to zero.

$$z'_i(v) = \begin{cases} -\infty, & v \in G_i, \\ z_i(v), & v \notin G_i \end{cases}$$

Final sampling for next token comes from $P_i(v) = softmax(z'_i(v))$. This rule gives a strong and explicit way to encode the target colour relation into the sampled sequence.

## 4.3. Detection

### 4.3.1 Local Consistency Reconstruction

Given candidate sequence, the detector reconstructs the same greenlists $G_i$ using the same local-prefix rule and secret key. For each eligible position $i \geq w$ with $b_i > 0$, compute:

1. Anchor colour under the current greenlist $c_{a(i)}^{(i)} = 1[x_{a(i)} \in G_i]$
2. Token colour under the same greenlist $c_i^{(i)} = 1[x_i \in G_i]$

Defines a local consistency indicator $u$ where $u_i = 1$ means that position $i$ satisfies the relation prescribed by the pattern bit.

$$u_i = \begin{cases} 1[c_i^{(i)} = c_{a(i)}^{(i)}], & r_{b_i} = 1, \\ 1[c_i^{(i)} \neq c_{a(i)}^{(i)}], & r_{b_i} = 0 \end{cases}$$

### 4.3.2 Global Consistency Statistic

Across all tested positions, the detector forms the aggregate statistic:

$$S = \sum_{i=1}^{n} u_i$$ where $n$ is the number of tested non-anchor positions.

If no watermark is applied, the same and opposite colour events are approximately balanced, resulting in expected $Pr(u_i = 1) \approx 0.5$. This gives the approximate null model $S \sim Binom(n, 0.5)$ with a corresponding global z-score of

$$Z_{cons} = \frac{S - 0.5n}{\sqrt{0.25n}}$$

When watermark is applied, we expect consistency to rise significantly beyond 0.5.

### 4.3.3 Block Voting with Phase Search

In practice, to tolerate alignment drift from paraphrasing, insertions, or deletions, the detector performs a phase search over all candidate offsets $\phi \in \{0, 1, \ldots, B - 1\}$

For fixed phase $\phi$, divide the consistency sequence into non-overlapping blocks of size $B$ i.e. $(u_{\phi}, u_{\phi+1}, \ldots, u_{\phi+B-1})$, $(u_{\phi+B}, u_{\phi+B+1}, \ldots, u_{\phi+2B-1})$,...

For each block $l$, define the number of matches as:

$$M_l = \sum_{j=1}^{B} u_{l,j}$$

A block is counted as a hit if its match count exceeds a threshold $m_{min}$. Let $H_{\phi}$ be the number of hit blocks under phase $\phi$ and $L_{\phi}$ be the total number of tested blocks. Under null, each $u_i$ behaves approximately as Bernoulli$(0.5)$. The null probability that a block is a hit is:

$$p_0 = \sum_{k=m_{min}}^{B} \binom{B}{k} w^{-B}$$

Therefore, $H_{\phi} \sim Binom(L_{\phi}, p_0)$ approximately under the null. For each phase, the detector computes the one-sided p-value

$$p_{\phi} = Pr(Binom(L_{\phi}, p_0) \geq H_{\phi})$$

Since multiple phases are searched, correction factor $c = max(1, log_2 B)$ is applied, yielding adjusted p-value $p_{\phi}^{adj} = min(1, cp_{\phi})$ to ensure that the final p-value is less likely to be significant because one phase appears consistent by chance. The detector selects the best phase with lowest adjusted p-value:

$$\phi^* = argmin_{\phi} p_{\phi}^{adj}$$

### 4.3.4 Final Detection Statistic

For selected phase $\phi^*$, define the block-level z-score

$$Z_{block} = \frac{H_{\phi^*} - L_{\phi^*} \cdot p_0}{\sqrt{L_{\phi^*} \cdot p_0 (1 - p_0)}}$$

Define reported z-score as

$$Z_{final} = max(Z_{cons}, Z_{block}).$$

Since we are combining two statistics, raising the z-threshold is needed to compensate for multiple opportunities to trigger detection, reducing false positives at the cost of some sensitivity (Jafari & Ansari-Pour, 2019).

Detection returns positive if $Z_{final}$ exceeds the Bonferroni-corrected threshold $Z_{Bonf}$.

# 5. Experimental Setup

PCM is compared against two established watermarking approaches: Baseline Zero-bit Watermark and Multi-bit Watermark. The three methods span a spectrum of design choices within the same language-model generation pipeline.

Evaluating these three together therefore isolates the central trade-offs of interest in watermark research: zero-bit/multi-bit design, global/local dependency, and token-level/block-wise detection. Since all three operate on the same model, prompts, attacks, and evaluation metrics, their comparison provides a fair and informative assessment of how different watermarking principles affect detectability, robustness, and text quality.

## 5.1 Dataset, Model Selection, and Hyperparameters

The evaluation is conducted on the large-scale WikiText-103 dataset, comprising approximately 103 million tokens from high-quality Wikipedia articles. Due to computational constraints, for each evaluation run, we sample 80-100 prompts uniformly at random from the test split. Depending on the number of sampled prompts, generated texts can contain $T$ tokens, where $T \in \{400, 600\}$. GPT-2 is used as the base language model, evaluated in inference mode with no gradient updates running on CPU. For all generation tasks, sampling temperature is fixed to 0.75 to balance stability and diversity. Token generation proceeds sequentially with the model's default vocabulary size $N = 50,257$.

To ensure fair comparison, all three watermarking methods are uniformly evaluated under a standardised set of hyperparameters in Appendix A.2. While optimal thresholds may improve PCM further, hyperparameter tuning is beyond the scope of this project, hence we leave this to future work. Across all models, the secret $K$ is derived from a fixed seed common to generator and detector, enabling reproducibility.

## 5.2 Attack Strategies

We assume a post-generation text adversary whose goal is to remove or weaken the watermark signal while preserving semantic content and readability. The adversary has white-box access to the generated text but does not know the secret watermark key, detector seed, or internal detector parameters. They may know the watermark family and general algorithm class, and can apply unrestricted text transformations such as paraphrasing, token shuffling, and deletion under a bounded distortion budget.

In our primary setting, the adversary cannot query the detector, which models offline redistribution/editing scenarios. Under this model, robustness is defined as maintaining high separability between watermarked and non-watermarked text after adversarial edits. Watermark robustness is then evaluated under three realistic text perturbation attacks:

| **Attack** | **Description** |
|---|---|
| **Paraphrase** | Randomly rephrase each tokens in the generated sequence with probability $p = 0.2$ using words from a small selection pool. Represent human rewriting or word-by-word paraphrasing and does not preserve token-for-token fidelity. |
| **Deletion** | Randomly delete tokens from the generated sequence with probability $p = 0.1$ to test for resilience to significant information loss. |
| **Shuffle** | Randomly permute the order of sentences with probability $p = 0.5$ to disrupt sequential dependencies while keeping lexical content intact. |

*Table 2.* Description of Attacks

For each attack, we generate 3 variants per prompt and compute metrics across all attack instances. Attack parameters are fixed to ensure comparability.

## 5.3 Evaluation Metrics

Performance is reported across four metric categories. Detection performance is measured using area-under-curve (AUC), which evaluates how well each detector separates watermarked samples from non-watermarked controls across all decision thresholds. Detection is run after each attack and the resulting AUC is reported. A robust scheme maintains high AUC even after perturbations. Text quality is evaluated with distinct scores and perplexity. Efficiency is measured using end-to-end generation latency, alongside token throughput per second. Calibration is assessed via the Brier proxy – an extension of the Brier score, designed to measure internal confidence-decision consistency.

The full metric table with definitions included is provided in Appendix A.2.

# 6. Evaluation

We report two main experimental settings: a primary benchmark with 100 prompts and 400-token generation budget, and a length-sensitive benchmark with 80 prompts and 600-token budget. Across both settings, we compare Baseline, Multi-bit, and PCM under clean generation and three perturbation attacks, namely paraphrase, shuffle, and deletion.

## 6.1 Robustness

### 6.1.1 PCM vs. Baseline

| **AUC Baseline vs. PCM – 400 tokens** | | |
|---|---|---|
| | **Baseline** | **PCM** |
| Clean | **0.9809** | 0.9787 |
| Attacked – Overall | **0.9383** | 0.6482 |
| Attacked – Paraphrase | **0.9283** | 0.6223 |
| Attacked – Shuffle | **0.9452** | 0.6474 |
| Attacked – Deletion | **0.9416** | 0.6750 |

*Table 3.* Baseline vs. PCM – 400 tokens

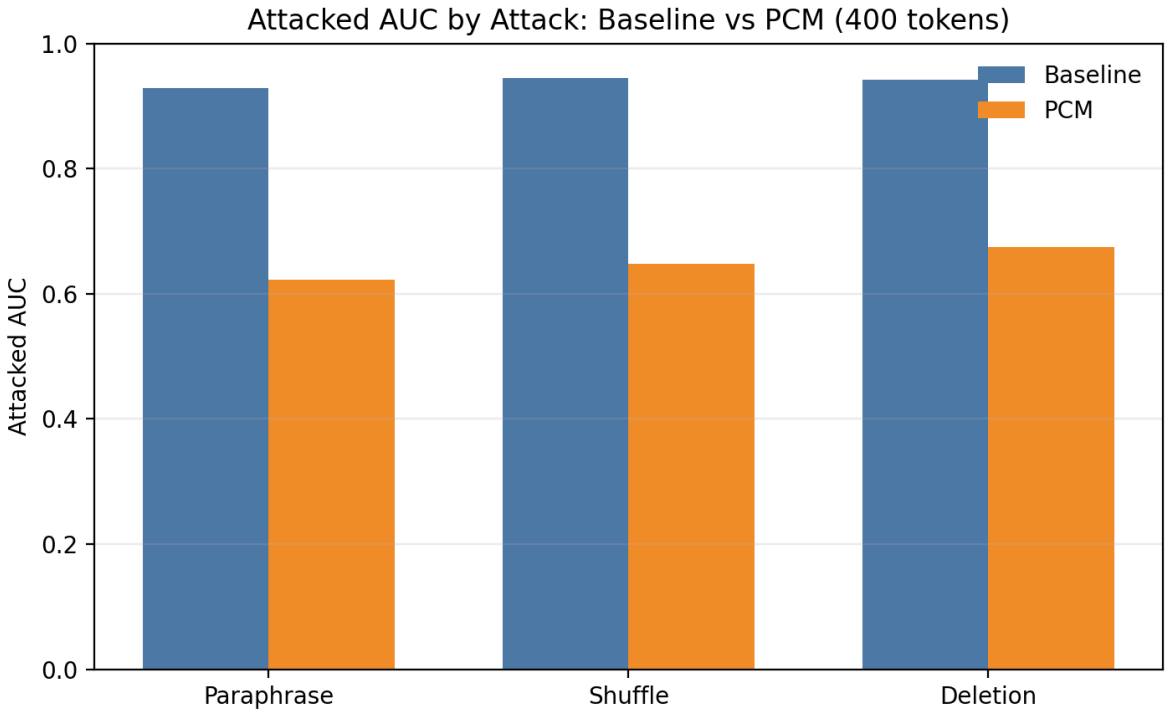



*Figure 1.* Baseline vs. PCM – 400 tokens

At 400 tokens, both methods show near-ceiling clean detection performance, with clean AUCs of 0.9809 (Baseline) and 0.9787 (PCM). Under attack, however, Baseline is substantially more robust, with attacked AUC 0.9383 versus 0.6482 for PCM. The same trend holds across individual attacks, where Baseline exceeds PCM on paraphrase (0.9283 vs. 0.6223), shuffle (0.9452 vs. 0.6474), and deletion (0.9416 vs. 0.6750).

| **AUC Baseline vs. PCM – 600 tokens** | | |
|---|---|---|
| | **Baseline** | **PCM** |
| Clean | 0.9900 | **0.9909** |
| Attacked – Overall | **0.9562** | 0.7049 |
| Attacked – Paraphrase | **0.9685** | 0.6466 |
| Attacked – Shuffle | **0.9139** | 0.7515 |
| Attacked – Deletion | **0.9863** | 0.7166 |

*Table 4.* Baseline vs. PCM – 600 tokens

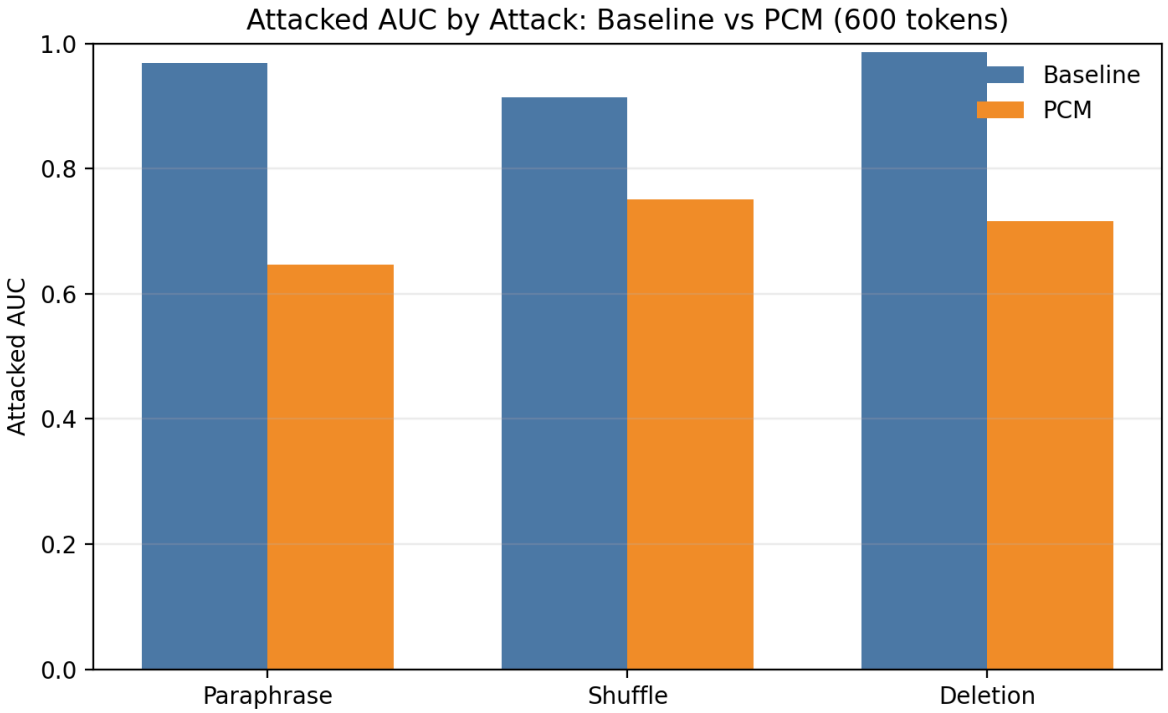



*Figure 2*. Baseline vs. PCM – 600 tokens

At 600 tokens, both methods remain near-perfect on clean detection (Baseline 0.9900, PCM 0.9909), and under attack Baseline is still more robust overall (0.9562 vs. 0.7049). Relative to 400 tokens, however, the attacked robustness gap narrows from 0.2901 (0.9383 vs. 0.6482) to 0.2513 (0.9562 vs. 0.7049), indicating that PCM strengthens with longer continuations even though it does not overtake Baseline.

The narrowing is most pronounced for shuffle: PCM rises from 0.6474 to 0.7515 while Baseline decreases from 0.9452 to 0.9139, reducing the shuffle gap from 0.2978 to 0.1624. By contrast, paraphrase and deletion mprovements are marginal, so the main length-driven gain for PCM is concentrated in reordering-style perturbations.

### 6.1.2 PCM vs. Multi-bit

| AUC Multi-bit vs. PCM – 400 tokens | | |
|---|---|---|
| | **Multi-bit** | **PCM** |
| Clean | 0.9277 | **0.9787** |
| Attacked – Overall | **0.8224** | 0.6482 |
| Attacked – Paraphrase | **0.7715** | 0.6223 |
| Attacked – Shuffle | **0.8330** | 0.6474 |
| Attacked – Deletion | **0.8628** | 0.6750 |

*Table 5.* Multi-bit vs. PCM – 400 tokens

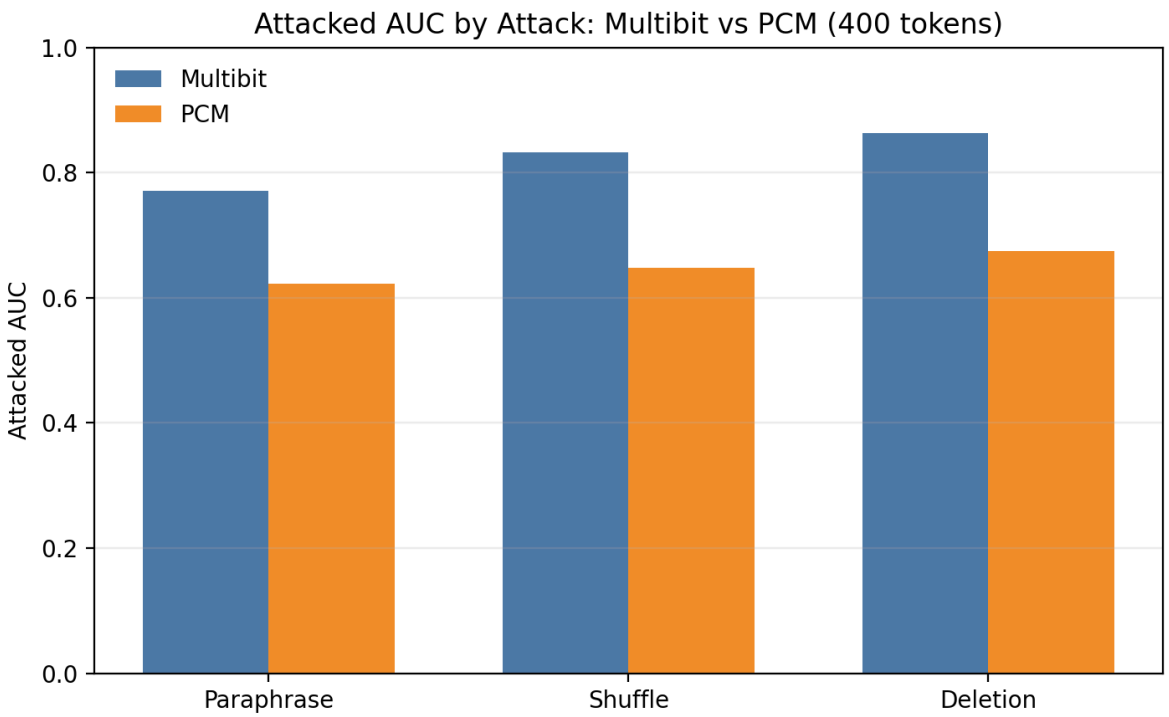



*Figure 3.* Multi-bit vs. PCM – 400 tokens

At 400 tokens, despite strong clean AUC (Multi-bit 0.9277 and PCM 0.9787), attacked results favours Multi-bit with overall attacked AUC 0.8224 vs. 0.6482 for PCM. The same pattern appears across individual attacks, where Multi-bit exceeds PCM on paraphrase (0.7715 vs 0.6223), shuffle (0.8330 vs 0.6474), and deletion (0.8628 vs 0.6750).

| AUC Multi-bit vs. PCM – 600 tokens | | |
|---|---|---|
| | **Multi-bit** | **PCM** |
| Clean | 0.9166 | **0.9861** |
| Attacked – Overall | **0.8254** | 0.7049 |
| Attacked – Paraphrase | **0.8250** | 0.6466 |
| Attacked – Shuffle | **0.8233** | 0.7515 |
| Attacked – Deletion | **0.8278** | 0.7166 |

*Table 6.* Multi-bit vs. PCM – 600 tokens

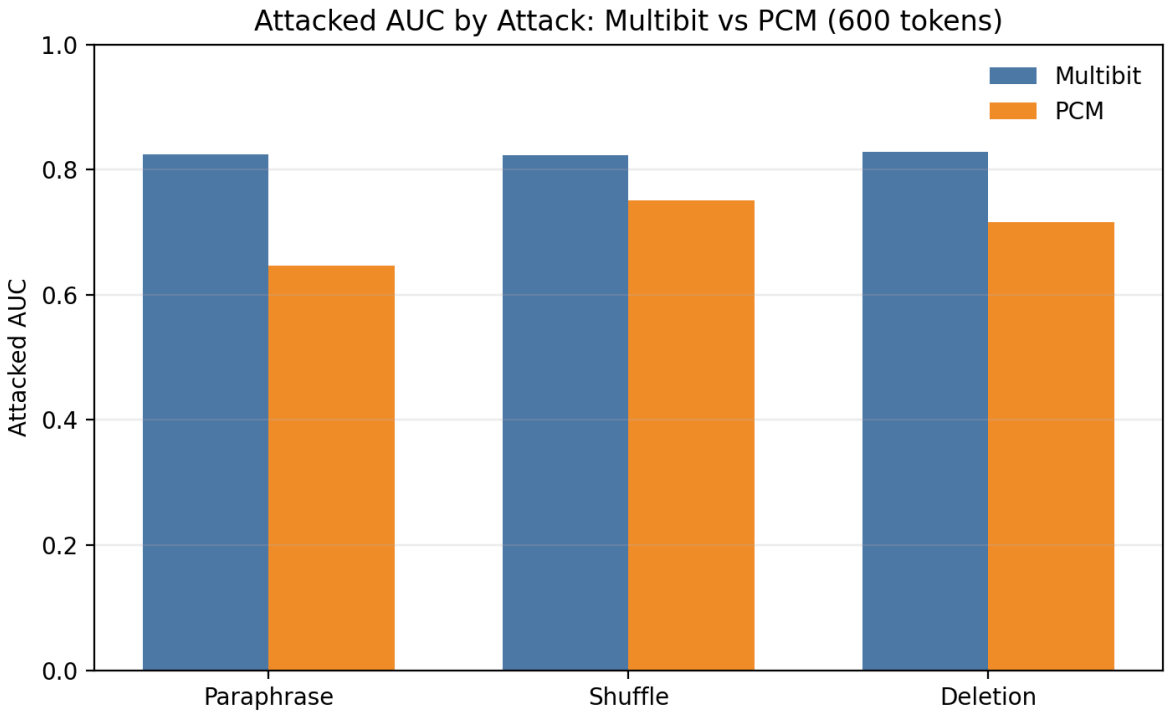



*Figure 4.* Multi-bit vs. PCM – 600 tokens

At 600 tokens, the ranking remains in favour of Multi-bit, but the robustness gap with PCM is smaller than at 400 tokens. Multi-bit still has higher attacked overall AUC (0.8254 vs 0.7049), yet the margin narrows from 0.1742 at 400 tokens to 0.1205 at 600 tokens. As similarly observed in Section 6.1.1, this convergence is driven mainly by shuffle robustness: PCM improves substantially (0.6474 to 0.7515), reducing the shuffle gap from 0.1856 to 0.0718. By contrast, paraphrase (0.6466) and deletion (0.7166) changes for PCM are modest.

### 6.1.3 Length Sensitivity

A key pattern in the updated results is that PCM attacked overall AUC increases most significantly with longer continuations, from 0.6482 at 400 tokens to 0.7049 at 600 tokens, while Multi-bit (0.8224 to 0.8254) and Baseline (0.9383 to 0.9562) also improve slightly. Nevertheless, the absolute ranking does not flip at longer length: Baseline remains strongest, Multi-bit second, and PCM third.

However, the pairwise gaps show a more nuanced trend. PCM narrows its gap to Baseline as difference drops from 0.2901 to 0.2513, and to Multi-bit from 0.1742 to 0.1205. This indicates that PCM gains more from additional context than its 400-token score alone suggests. The improvement is especially visible under shuffle attacks, where PCM rises from 0.6474 to 0.7515 and closes much of the margin to Multi-bit. This is consistent with the intuition that longer sequences provide more opportunities for relational evidence to accumulate, even though, in this configuration, that gain is not yet sufficient for PCM to overtake the stronger baselines on attacked robustness.

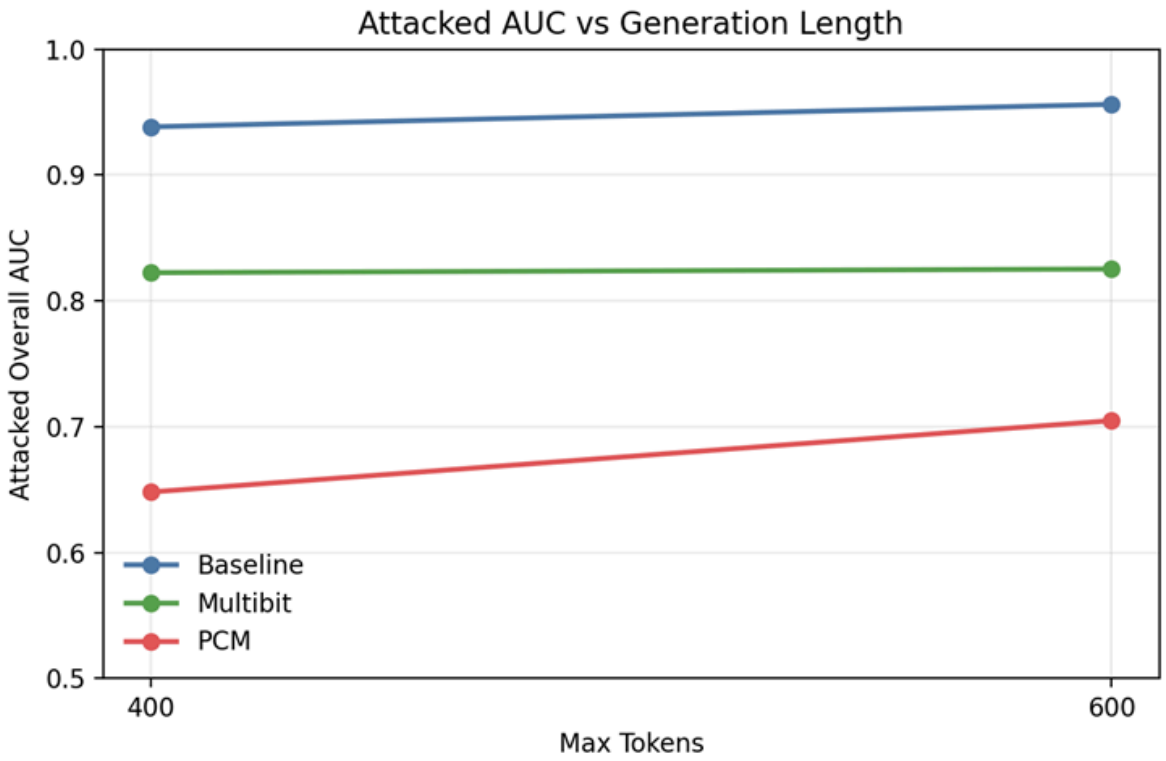



*Figure 5.* Attacked AUC at Generation Lengths 400 and 600 tokens

## 6.2. Text Quality

| Clean Text Quality Baseline vs. Mult-bit vs. PCM | | | | | |
|---|---|---|---|---|---|
| | | **Unwatermarked** | **Baseline** | **Multi-bit** | **PCM** |
| **400-token** | Perplexity | 18.582 | **10.401** | 22.246 | 22.115 |
| | Distinct-1 | 0.2118 | 0.2289 | 0.2233 | **0.2668** |
| | Distinct-2 | 0.5534 | 0.5864 | 0.5927 | **0.6971** |
| **600-token** | Perplexity | 19.024 | **8.491** | 11.241 | 18.239 |
| | Distinct-1 | 0.1677 | 0.1947 | 0.2103 | **0.2534** |
| | Distinct-2 | 0.4531 | 0.5221 | 0.5770 | **0.6917** |

*Table 7*. Clean Text Quality Comparison

PCM delivers the highest lexical diversity (400-token: Distinct-1 0.2668, Distinct-2 0.6971 vs. 600-token: Distinct-1 0.2534, Distinct-2 0.6917), above Baseline (0.2289 and 0.5864 and 0.1947 vs. 0.5221) and Multi-bit (0.2233 and 0.5927 vs. 0.2103 and 0.5770).

Regarding perplexity, Multi-bit is highest for 400-token (22.246) but suppressed at 600-token (11.241), while PCM falls from 22.115 to 18.239, and Baseline is much lower at 10.401 and 8.491. This suggests a potential diversity–fluency tension primarily born by PCM.

| Statistical Calibration Baseline vs. Mult-bit vs. PCM | | | | |
|---|---|---|---|---|
| | | **Baseline** | **Multi-bit** | **PCM** |
| **400-token** | Mean p-value | **0.0003** | 0.0121 | 0.0367 |
| | Brier proxy | **0.0000** | 0.0892 | 0.0089 |
| **600-token** | Mean p-value | **0.0049** | 0.0073 | 0.0179 |
| | Brier proxy | **0.0047** | 0.0268 | 0.0061 |

*Table 8*. Statistical Calibration Comparison

For both runs, calibration strongly favours Baseline: it has the lowest mean p-value (0.0003/0.0049) and the lowest Brier proxy (0.0000/0.0047), while Multi-bit reports 0.0121/0.0073 and 0.0892/0.0268, and PCM reports 0.0367/0.0179 and 0.0089/0.0061. By this proxy, Baseline shows the strongest internal confidence-decision consistency, PCM intermediate, and Multi-bit is the least consistent.

| Efficiency Baseline vs. Mult-bit vs. PCM | | | | |
|---|---|---|---|---|
| | | **Baseline** | **Multi-bit** | **PCM** |
| **400-token** | Mean latency (s) | 99.98 | 97.57 | **83.50** |
| | Mean tokens/s | 3.801 | 3.739 | **4.477** |
| **600-token** | Mean latency (s) | 137.26 | 140.87 | **111.88** |
| | Mean tokens/s | 3.875 | 3.801 | **4.548** |

*Table 9*. Efficiency Comparison at 400 tokens

Efficiency favours PCM, which has the lowest mean latency and highest throughput (83.50/111.88s, 4.477/4.548 tokens/s), ahead of Baseline (99.98/137.26s, 3.801/3.875 tokens/s) and Multi-bit (97.57/140.87s, 3.739/3.801 tokens/s). Note that efficiency gains are implementation-specific and may not generalise to optimised GPU/TPU kernels.

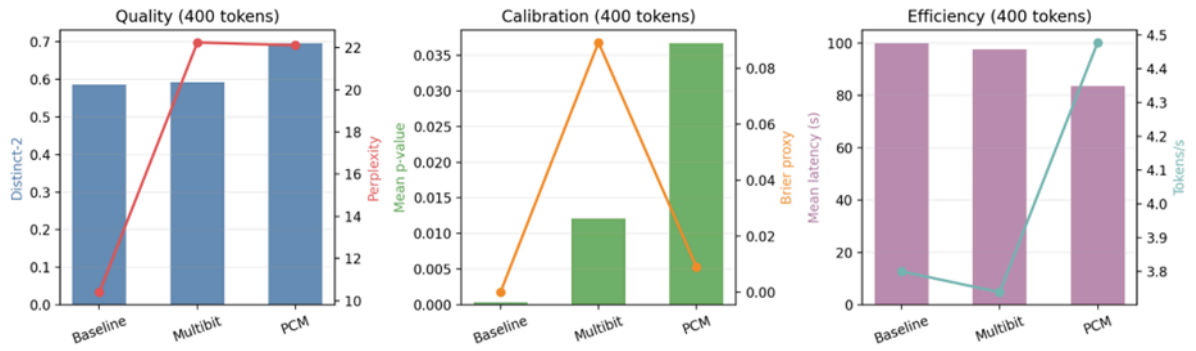



*Figure 6.* Quality, Calibration, and Efficiency at 400 tokens

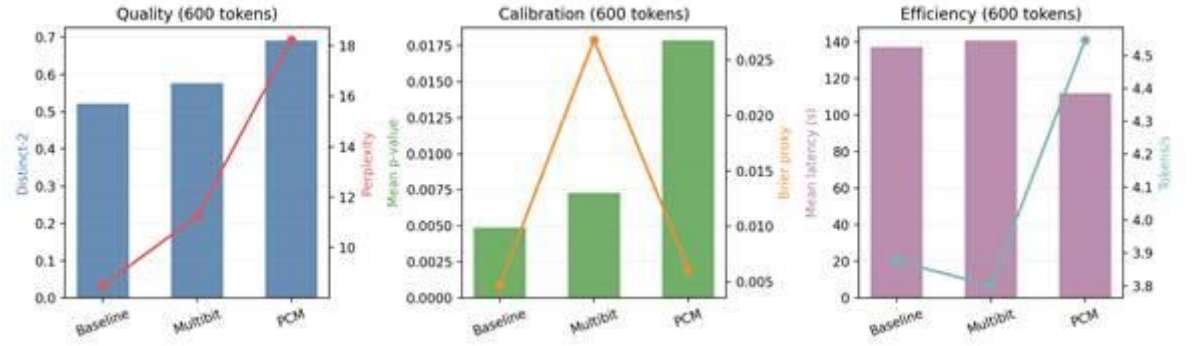



*Figure 7.* Quality, Calibration, and Efficiency at 600 tokens

## 6.3 Summary of Findings

All methods are highly detectable on clean text, so clean AUC is not the key separator. The main differentiator is post-attack reliability, where Baseline is strongest with overall attacked AUC 0.9383 at 400 tokens and 0.9562 at 600 tokens, followed by Multi-bit (0.8224 and 0.8254), while PCM remains lower (0.6482and 0.7049). However, PCM is still substantially above chance and retains meaningful detection potential under perturbation. Importantly, PCM improves with length, rising from 0.6482 at 400 tokens to 0.7049 at 600 tokens, which indicates recoverable signal strength as more evidence accumulates.

Calibration by the reported proxy favours Baseline, but PCM offers the best runtime profile with lowest latency and highest throughput and the strongest lexical diversity, suggesting a viable foundation with clear room for robustness-oriented refinement.

# 7. Discussion

**Clean detectability is necessary but not sufficient.** Under the 400-token and 600-token runs, all three methods remain strongly detectable on unmodified text, but PCM is strongest on clean detection quality at longer length and on clean accuracy at both lengths. At 600 tokens, PCM achieves the highest clean AUC and accuracy (AUC 0.9909, accuracy 0.9563), ahead of Baseline (0.9900, 0.9063) and Multi-bit (0.9166, 0.8688). At 400 tokens clean AUC is still high across methods (Baseline 0.9809, Multi-bit 0.9277, PCM 0.9787), while PCM records the highest clean accuracy (0.9350). This remains a useful sanity check, but in deployment the key criterion is post-edit reliability whether watermark evidence survives paraphrasing, reordering, and deletion.

**Contrary to our original hypothesis, PCM fails to maintain robustness under perturbations.** Under attack at 400 tokens, PCM is the weakest method (overall attacked AUC 0.6482), well below Multi-bit (0.8224) and Baseline (0.9383). The same ranking holds across all attack types: paraphrase (PCM 0.6223 vs. Multi-bit 0.7715 vs. Baseline 0.9283), shuffle (0.6474 vs. 0.8330 vs. 0.9452), and deletion (0.6750 vs. 0.8628 vs. 0.9416).

At 600 tokens, the ordering remains unchanged from the 400-token run: Baseline is highest, Multi-bit is second, and PCM is third in attacked robustness. Specifically, at 600 tokens, attacked mean AUC is 0.9562 (Baseline), 0.8254 (Multi-bit), and 0.7049 (PCM). Attack-wise, the same ranking holds for paraphrase (0.9685 > 0.8250 > 0.6466), shuffle (0.9139 > 0.8233 > 0.7515), and deletion (0.9863 > 0.8278 > 0.7166).

**However, the robustness gaps observed in PCM narrow with length.** PCM improves from 0.6482 to 0.7049 overall attacked AUC, reducing its gap to Baseline (from 0.2901 to 0.2513) and to Multi-bit (from 0.1742 to 0.1205). The largest convergence appears under shuffle, where PCM increases from 0.6474 to 0.7515 and closes much of the margin to Multi-bit. This supports a length-scaling interpretation that still favours PCM: longer continuations provide more relational events and block-level comparisons, allowing weak local consistency signals to accumulate into a stronger aggregate decision, even if not yet enough to overtake stronger baselines in this configuration.

**PCM was expected to gain robustness from dual evidence aggregation, but that advantage did not materialise.** The key reason is that PCM is structurally a relational detector, while the attacks in this benchmark are exactly the kinds of transformations that destabilise token-level relations. Baseline, by contrast, relies on a simpler marginal signal, and in this setting that simplicity is a strength rather than a weakness.

At a design level, PCM depends on multiple alignment assumptions being simultaneously valid: warmup boundary, block index, phase alignment, anchor position, and prefix-conditioned color assignment. Its detector computes consistency and block statistics on top of those assumptions, then combines them through $Z_{final} = max(Z_{cons}, Z_{block})$.

This sound appealing in theory because there are two paths to detection. However, as demonstrated through our experiments, both paths can degrade together when edits alter local neighborhoods and relative positions. Paraphrasing changes lexical realisations that feed prefix hashes, while deletion and shuffle disrupt the positional structure that block logic relies on. So the complementary statistics are not fully independent failure modes; they often share the same upstream fragility.

**A central structural disadvantage is synchronisation sensitivity.** PCM's evidence is extracted from block-organised non-anchor positions after a fixed warmup, so edits that insert, remove, or reorder text can desynchronise token-to-block mapping. Once synchronisation drifts, both consistency counting and block-hit accumulation are computed on partially misregistered events. Even if semantic content remains similar, the detector may be comparing the wrong relational pairs. Baseline does not require this degree of positional synchronisation: it aggregates a global green-membership surplus, which is less brittle to local alignment errors.

**Detector conservativeness further contributes.** With Bonferroni-adjusted thresholding, PCM reduces false positives but also raises the bar for positive calls, which can hurt recall under attack when signal quality is already degraded. Baseline's statistic, combined with current configuration choices, appears to preserve enough separation under the same attacks to maintain much higher attacked AUC. So the updated outcome is not merely PCM underperformed; it suggests a mismatch between PCM's structural assumptions and the perturbation model used here.

**The broader implication is that PCM's theoretical benefit is conditional.** It performs best when enough structural correspondence survives for its dual-statistic detector to accumulate consistent evidence. When alignment is heavily disrupted, this dependence can reduce robustness relative to simpler schemes. Nevertheless, **PCM behaves well above chance under attack**, **and shows measurable convergence toward stronger baselines at longer continuations.**

**PCM also displays strong efficiency signals.** At 400-token, it is the most efficient method at inference time, with the lowest mean latency (83.50s) and highest throughput (4.477 tokens/s), outperforming Baseline (99.98s, 3.801 tokens/s) and Multi-bit (97.57s, 3.739 tokens/s) by a large margin. The same is observed at 600 tokens, where PCM again has the lowest mean latency (111.88s) and highest throughput (4.548 tokens/s), outperforming Baseline (137.26s, 3.875 tokens/s) and Multi-bit (140.87s, 3.801 tokens/s).

The efficiency advantage stems directly from PCM anchor token generation logic. With block size 8, the first position of every block is an anchor and receives zero modification. PCM therefore only runs its logit processor on 7/8 of tokens, compared to Baseline and Multi-bit which run on every token without exception. This finding is empirical for our hardware implementation and may vary with optimised GPU kernels.

**In addition, PCM achieves high lexical diversity.** PCM scores highest in unigram and bigram diversity (Distinct-1 0.2668, Distinct-2 0.6971 at 400 tokens and Distinct-1 0.2534, Distinct-2 0.6917 at 600 tokens). The key to this preserved diversity is through its anchor design, where the first token in each block is an anchor and is not logit-modified, so every eighth position is sampled from the model's unforced distribution. These unmodified anchor positions act as periodic free generation points that help maintain lexical spread even when non-anchor positions are constrained.

By contrast, Baseline's lower diversity scores are consistent with stronger shortlist reuse and repetitive phrasing. In practical terms, this can make watermarked text easier to spot heuristically, because repeated templates and recurring phrase patterns become more visible to human readers even before formal detection is applied (Khachaturov et al., 2025).

**Furthermore, PCM has minimal fluency distortion relative to plain text.** At 400 tokens, PCM perplexity is 22.11, which is closer to plain-text perplexity (18.58) (see Appendix A.3) than Multi-bit (22.25), while Baseline is unusually low (10.40), consistent with stronger distributional compression. The same pattern appears at 600 tokens: plain is 19.02 (see Appendix A.3), PCM is 18.24 (closest), Multi-bit is 11.24, and Baseline is 8.49. This supports the interpretation that persistent green/fixed-colour bias in Baseline and Multi-bit can steer decoding toward a constrained token subset and depress entropy, whereas PCM's block-anchor design applies logit pressure only to non-anchor positions, leaving every eighth token unconstrained and better preserving natural generation dynamics. We observe that PCM maintains a more plain-like fluency profile across both lengths, while still trading some perplexity movement for watermark signal under attack.

The elevated perplexity should therefore not be interpreted as a diversity-fluency penalty, but because of PCM's lighter per-token interference, as a less biased decoder preserves broader n-gram variation and stays closer to natural generation dynamics. This quality profile is directly relevant to stealth because watermark signals that preserve native lexical behaviour are less likely to produce repetitive or unnatural artefacts that humans or heuristic detectors can exploit. Taken together, PCM's diversity and near-clean perplexity indicate a more stealth-aligned watermarking behaviour in this comparison.

**Calibration is mixed under the reported proxy.** At 400 tokens, Brier proxy is lowest for Baseline (0.000005), followed by PCM (0.0089), and highest for Multi-bit (0.0892). At 600 tokens, the same pattern remains: Baseline (0.0047), PCM (0.0061), Multi-bit (0.0268). For governance and audit use, this matters because calibration quality affects how reliably thresholds translate into predictable error behaviour. While PCM shows stable middle-ground calibration across lengths, Baseline boasts optimised levels of calibration.

**These results place PCM in a similar design conversation to SynthID and SemaMark methods,** as they all seek watermark signals that are more compatible with natural text generation. Like those families, PCM benefits from a more structured signal that helps preserve generation quality, reflected here in strong diversity and close-to-clean perplexity. The difference is that PCM remains much lighter-weight operationally as it uses keyed greenlist partitioning and relational consistency checks, whereas SynthID relies on multidirectional signal shifts that require heavyweight analysis and SemaMark calls for discretisation or embedding-space reasoning over continuous semantic structure.

At the same time, PCM shares a broader limitation common to semantic- or structure-aware watermarking: once the signal depends on higher-order relations rather than simple marginal bias, robustness can become sensitive to edit-induced misalignment. PCM can be viewed as a lightweight relational alternative within the same broader research direction, with a practical efficiency advantage, but one that still requires stronger semantic-invariant design to remain robust under heavier perturbations.

**Overall,** within the evaluated regime, PCM offers the best runtime efficiency and lexical diversity for an upstanding stealth posture. While robustness is limited in current iteration, AUC performance exceeds chance level by a notable margin and improves with length. This leaves substantial room for improvement rather than invalidating the approach. In that sense, PCM should be treated as a promising extension of the classical greenlist framework as it already preserves key practical strengths through efficiency and text quality while offering a structured foundation that can be further hardened for post-edit robustness.

The primary research opportunity is to design structural, semantic-preserving watermarks that retain relational advantages without brittleness to edits. Promising directions include anchor schemes tied to semantic units such as clauses or sentences, edit-invariant synchronisation markers, multi-scale detectors that combine local lexical and sentence-level semantic consistency, and redundancy coding that preserves recoverability under paraphrase and reordering. The goal should be a detector that tracks meaning-preserving transformations, not only token-level alignment, so robustness improves without sacrificing efficiency.

# 8. Limitations

**This study is constrained to a single-model, single-domain evaluation setting:** GPT-2 (124M) with Wikipedia-derived prompts, using limited token counts and generation budget at 100 prompts, 400 generated tokens and 80 prompts, 600 generated tokens. This design supports reproducibility and controlled analysis, but it limits external validity. The observed behavior, especially the gap between clean detectability and attacked performance, may change under larger decoder-only models, instruction-tuned checkpoints, or specialised corpora with different lexical and syntactic statistics.

A broader multi-model evaluation across scale, alignment regimes, and domains is required before making strong generalisation claims.

**A second limitation is the fixed structural configuration of each watermark implementation.** In the current setup, performance is sensitive to concrete design choices such as prefix window size, block structure, and thresholding rules. This means conclusions are partly configuration-dependent rather than purely method-intrinsic. In particular, methods that rely on positional or relational structure like PCM can be vulnerable to synchronisation drift under edits, while simpler global statistics may behave differently under the same perturbations.

Future work should treat structural settings as first-class variables and report ablations over context width, block size, warmup, and detector aggregation strategy.

**The threat model is intentionally controlled and non-adaptive.** Attacks are limited to paraphrase (0.2), shuffle (0.5), and deletion (0.1), applied independently. These transformations are useful for reproducible stress testing, but they do not represent a worst-case adversary. Scheme-aware attackers could optimise edits to suppress detector evidence while preserving semantics, for example by targeting local context cues, boundary alignment, or high-signal spans.

Extending the attack suite to include stronger paraphrase models, back-translation, adversarially optimised edits, and compositional attack chains is a natural and important continuation of this work.

# 9. Conclusion

This project introduced PCM (Pairwise Consistency Matching) as a zero-bit semantic-preserving watermarking extension of the Kirchenbauer greenlist paradigm. PCM imposes keyed anchor-conditioned relational rules within blocks and aggregates evidence through a composite detector with corrected thresholding, extending greenlist watermarking from frequency bias toward structure-aware verification.

Across the evaluated settings, all methods remain highly detectable on clean text, while attacked robustness results are mixed. PCM does not rank highest under perturbation in this benchmark, but it remains clearly above chance and therefore retains meaningful detection utility under attack. Importantly, PCM improves witlength, indicating that additional continuation context helps its relational signal accumulate more effectively. This suggests not failure of the paradigm, but room for targeted improvement. PCM also shows consistent practical strengths: fastest decoding profile, highest lexical diversity, and perplexity behaviour closest to plain generation. These properties position PCM as a promising quality- and efficiency-oriented extension of classical greenlist watermarking, with a credible path to stronger robustness through further refinement.

This framing aligns PCM with semantics-aware watermarking directions such as SynthID and SemaMark. Within that landscape, PCM contributes a lightweight, retraining-free design that preserves the elegance of keyed greenlist methods while moving toward semantic preservation. The central open challenge remains shared across these approaches: maintaining detectability under stronger meaning-preserving edits.

For practical deployment, watermarking should be approached as a full system problem, requiring in-domain threshold calibration, secure key lifecycle management, confidence-tiered decision policies, and routine adversarial stress testing beyond fixed simulation settings. The most important next step is to develop structural, semantic-preserving watermark variants that retain PCM's efficiency while improving robustness to edit operations such as paraphrase, reordering, and deletion. Promising directions include edit-invariant synchronisation, adaptive block/context scheduling, stronger calibration under mixed clean-attacked distributions, and hybrid schemes that combine relational zero-bit detection with lightweight payload channels.

# Appendix

## A.1 Full Algorithms

### A.1.1. PCM Watermark Generation

```
Input: language model, key K, warm-up w, block size B, green ratio γ
Output: watermarked sequence x_0:T-1

for each generation step i do
    if i < w then
        sample x_i normally
        continue
    end if

    m_i = i - w
    b_i = m_i % B

    if b_i = 0 then
        sample x_i normally # first token of block becomes anchor
        continue
    end if

    a_i = i - b_i
    p_i = last up to 4 tokens before position i
    G_i = keyed greenlist from H(K || p_i)

    anchor_colour = 1[x_a_i ∈ Gi]
    target_colour =
        anchor_colour        if r_b_i = 1
        1 - anchor_colour    if r_b_i = 0

    if target_colour = 1 then
        add ln 2 to logits of all v ∈ G_i
    else
        set logits of all v ∈ G_i to -∞
    end if

    sample x_i from softmax of modified logits
end for
```

### A.1.2. PCM Watermark Detection

```
Input: candidate sequence x_0:T-1, key K, warm-up w, block size B
Output: detected / not detected, z-score

Initialize consistency list U = [ ]

for each token position i ≥ w do
    m_i = i - w
    b_i = m_i % B

    if b_i = 0 then
        continue # skip anchor positions
    end if

    a_i = i - b_i
    p_i = last up to 4 tokens before position i
    G_i = keyed greenlist from H(K || p_i)

    anchor_colour = 1[x_a_i ∈ G_i]
    token_colour  = 1[x_i ∈ G_i]

    if r_b_i = 1 then
        u_i = 1[token_colour = anchor_colour]
    else
        u_i = 1[token_colour ≠ anchor_colour]
    end if

    append u_i to U
end for

Compute global consistency statistic S = sum(U)
Compute Z_cons from Binomial(|U|, 0.5)

best_p = 1
for phase φ in {0, 1, ..., B-1} do
    partition U into length-B blocks starting at φ
    count H_φ = number of blocks with at least m_min matches
    let L_φ = number of tested blocks

    compute p_φ under Binomial(L_φ, p_0)
    apply phase correction to obtain p_φ_adj

    keep phase with smallest adjusted p-value
end for

Compute Z_block for best phase
Set Z_final = max(Z_cons, Z_block)

Return detected if Z_final > z_threshold
```

### A.1.3: Baseline Watermark Encoding (Kirchenbauer et al., 2023)

```
Input: prompts s(-N_p)...s(-1)
    PRF F with key K
    hardness parameter delta > 0
    window width h > 0

for t = 0, 1,... do
   1. Apply the language model to s(-Np)...s(t-1) to get a logit vector
      l(t) over the vocabulary
   2. Sort the vocabulary so l(t) is in descending order. Set k = 0, the
      index of the most likely token.
   3. Temporarily set s(t) to be the k_th token in the vocabulary. Compute
      H_i = F_K(s(t),s(t-i)) for 1 ≤ i ≤ h.
   4. Set i* = argmin_i(H_i)
   5. Using H_i* as a seed, produce a random bit to decide if token k is on
      the green or red list.

    if greenlist is chosen then
        keep s (t) and continue.
    else if red list is chosen and l(t)_k+1 < l(t)_0 - delta then
        choose s (t) to be the most likely (k = 0) token, which is in the
        red list, and continue.
    else
        set k ← k + 1, goto to step 3.
    end if
end for
```

### A.1.4: Baseline Watermark Decoding (Kirchenbauer et al., 2023)

```
Input: watermarked text X_1:T, hash context width h, secret key K
Output: z-score Z, detection decision d ∈ {0, 1}

# initialise counter
W_green = 0

for t in [h + 1, T] do
    s = hash previous h tokens using K
    permute V_t using s as seed
    if X_t ∈ G_t then
        W_green += 1
    end if
end for

Calculate z-score Z for W_green
d = 1 if Z > z_threshold else 0

return Z, d
```

### A.1.5. Multi-bit Watermark Message Encoding (Yoo et al., 2024)

```
Input: prompts X_0:T-1
    hardness parameter delta > 0
    window width h > 0
    message m of effective length b

for t = 0, 1,... do
   1. Compute seed from previous context s = f(X_t-h:t-1)
   2. Sample position p = sample(b) using s as seed.
   3. Get message content m ← m[p]
   4. Permute vocabulary V_t using s as seed.
   5. Partition vocabulary into colourlists, discarding remainders
   6. Add delta to token logits in selected colourlist
   7. generate token
end for
```

### A.1.6. Multi-bit Watermark Message Decoding (Yoo et al., 2024)

```
Input: watermarked text X_1:T, hash context width h, effective length b
Output: predicted message m, number of colourlisted tokens w

# initialise counter
W_p[m] = 0 for all p, m

for t in [h + 1, T] do
    s = f(X_t-h:t-1)
    p = sample(b)
    for m in r do:
        permute V_t using s as seed
        if X_t ∈ G_m_t then
            W_p[m] += 1
        end if
    end for
end for

# predict message
m_r = ""
w = 0
for p in b do
    w += max (W_p[m])
    m = argmax_m(W_p[m])
    m_r += str(m)
end for

get bit message m by converting m_r
return m, w
```

## A.2 Hyperparameters and Evaluation Metrics

### A.2.1. Description of Hyperparameters

| Parameter | Model | Value | Justification |
|---|---|---|---|
| Greenlist ratio γ | All | 0.5 | Standard in prior works to balance red/green partition. |
| Logit bias δ | Baseline | 1.0 | Conservative bias consistent with baseline practice in prior work. |
| | Multi-bit | 2.0 | Stronger bias improves symbol recoverability for message decoding. |
| | PCM | | Strong bias to enforce anchor-conditioned colour constraints within blocks. |
| Prefix window | Baseline | 3 | Local context improves stability while preserving lightweight hashing (Kirchenbauer et al., 2023). |
| | Multi-bit | 1 | Position-wise symbol mapping uses immediate local context for fast updates (Yoo et al., 2024). |
| | PCM | 4 | Short rolling prefix supports anchor-consistency signal with moderate compute cost. |
| Detection z-threshold | Baseline | 1.96 | One-sided ($\alpha = 0.025$) significance level for a single test statistic. |
| | Multi-bit | | |
| | PCM | 2.24 | Bonferroni-corrected threshold to control Type I error when combining z-statistics. |

### A.2.2. Description of Evaluation Metrics

| Category | Metric | Description |
|---|---|---|
| **Robustness** | AUC | Area under the ROC curve across detected vs. non-detected samples over a range of decision thresholds. |
| **Text Quality** | Perplexity | Cross-entropy loss of the watermarked sequence under a reference non-watermarked GPT-2, normalised per token. Lower perplexity means better preservation of natural language statistics (Colla et al., 2022). |
| | Distinct-1 | Measures the unigram diversity in a sentence and penalises those with many repetitions (Li et al., 2016). |
| | Distinct-2 | Measures the bigram diversity in a sentence and penalises those with many repetitions (Li et al., 2016). |
| **Statistical Calibration**[1] | Mean p-value | Aggregated one-sided binomial p-value. Lower mean p-value indicates strong detection signal. |
| | Brier proxy | Mean squared gap between detector confidence $1 - \text{p-value}$ and the detector's binary decision. Lower values mean better internal confidence-decision consistency within detected watermarked texts, but it is not a true Brier score against ground-truth labels (Brier, 1950). |
| **Efficiency** | Latency | Per-request end-to-end generation latency from starting local generation to receiving the final generated sequence. This includes prompt processing, KV-cache prefill, token-by-token decoding, and watermark processing (Das et al., 2026). |
| | Tokens/s | Number of tokens generated per second. |

[1] All reported values are computed on watermarked samples only.

## A.3 Extended Results

### A.3.1 Clean Detection Performance

| | 400 tokens | | 600 tokens | |
|---|---|---|---|---|
| | **AUC** | **Accuracy** | **AUC** | **Accuracy** |
| Baseline | **0.9809** | 0.9250 | 0.9900 | 0.9063 |
| Multibit | 0.9277 | 0.8750 | 0.9166 | 0.8688 |
| PCM | **0.9787** | **0.9350** | **0.9909** | **0.9563** |

### A.3.2. Attacked AUC by Attack Type

| **Method** | **Token Length** | **Paraphrase** | **Shuffle** | **Deletion** | **Mean (95% CI)** |
|---|---|---|---|---|---|
| Baseline | 400 | **0.9283** | **0.9452** | **0.9416** | **0.938 [0.928, 0.945]** |
| | 600 | **0.9685** | **0.9139** | **0.9863** | **0.956 [0.914, 0.986]** |
| Multibit | 400 | 0.7715 | 0.8330 | 0.8628 | 0.822 [0.772, 0.853] |
| | 600 | 0.8250 | 0.8233 | 0.8278 | 0.825 [0.823, 0.827] |
| PCM | 400 | 0.6223 | 0.6474 | 0.6750 | **0.648 [0.622, 0.666]** |
| | 600 | 0.6466 | 0.7515 | 0.7166 | **0.705 [0.647, 0.751]** |

### A.3.3. Attacked Accuracy by Attack Type

| **Method** | **Token Length** | **Paraphrase** | **Shuffle** | **Deletion** | **Mean (95% CI)** |
|---|---|---|---|---|---|
| Baseline | 400 | **0.8600** | **0.9100** | **0.9000** | **0.890 [0.860, 0.910]** |
| | 600 | **0.9375** | **0.8875** | **0.9250** | **0.917 [0.887, 0.938]** |
| Multibit | 400 | 0.6400 | 0.7050 | 0.7700 | 0.705 [0.640, 0.770] |
| | 600 | 0.7188 | 0.7188 | 0.7688 | 0.735 [0.719, 0.769] |
| PCM | 400 | 0.5600 | 0.5800 | 0.5800 | 0.573 [0.560, 0.580] |
| | 600 | 0.5375 | 0.6313 | 0.6125 | 0.594 [0.537, 0.631] |

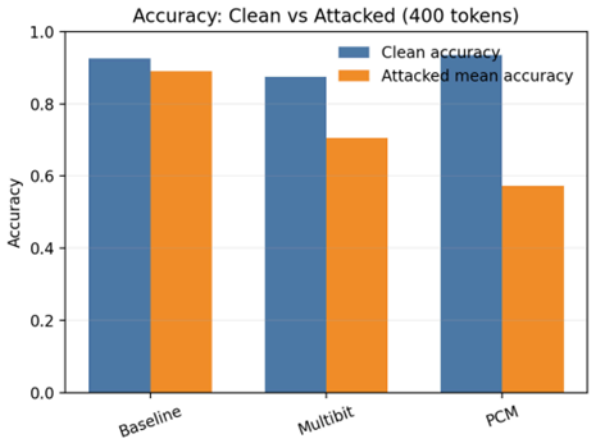
### A.3.4. Text Quality Metrics

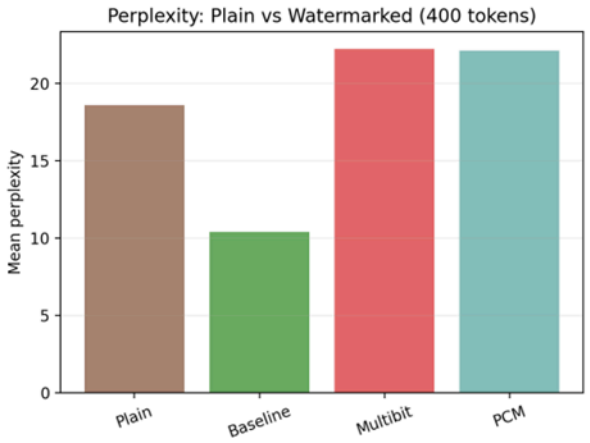
| Method | Token Length | Distinct-1 | Distinct-2 | Perplexity | BLEU vs. Plain |
|---|---|---|---|---|---|
| Plain | 400 | 0.2118 | 0.5540 | 18.58 | 1.0000 |
| | 600 | 0.1677 | 0.4531 | 19.02 | 1.0000 |
| Baseline | 400 | 0.2289 | 0.5864 | **10.40** | 0.0340 |
| | 600 | 0.1947 | 0.5221 | **8.49** | 0.0161 |
| Multibit | 400 | 0.2233 | 0.5927 | **22.25** | 0.0287 |
| | 600 | 0.2103 | 0.5770 | **11.24** | 0.0097 |
| PCM | 400 | **0.2668** | **0.6971** | 22.11 | 0.0295 |
| | 600 | **0.2534** | **0.6917** | 18.24 | 0.0123 |

### A.3.5. Calibration and Efficiency

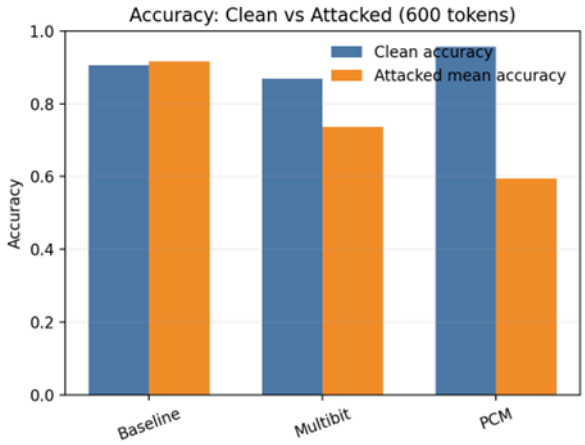
| Method | Token Length | Mean p-value | Brier proxy | Mean Latency (s) | p95 Latency (s) | Token/s |
|---|---|---|---|---|---|---|
| Baseline | 400 | **0.0003** | **0.000005** | 99.98 | 154.30 | 3.80 |
| | 600 | **0.0049** | **0.0047** | 137.26 | 224.65 | 3.88 |
| Multibit | 400 | 0.0121 | 0.0892 | 97.57 | 155.06 | 3.74 |
| | 600 | 0.0073 | 0.0268 | 140.87 | 237.62 | 3.80 |
| PCM | 400 | 0.0367 | 0.0089 | **83.50** | **137.96** | **4.48** |
| | 600 | 0.0179 | 0.0061 | **111.88** | **201.40** | **4.55** |

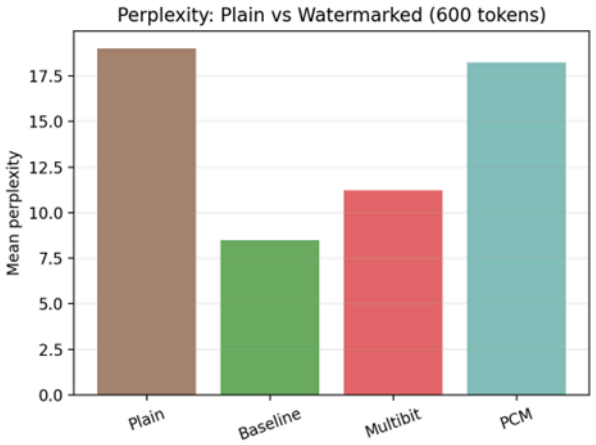
### A.3.6 Additional Plots – 400 tokens




### A.3.7 Additional Plots – 600 tokens



# References


Aickin, M., & Gensler, H. (1996). Adjusting for multiple testing when reporting research results: the Bonferroni vs Holm methods. *American Journal of Public Health*, *86*(5), 726–728. https://doi.org/10.2105/ajph.86.5.726

Brier, G. W. (1950). Verification of Forecasts Expressed in Terms of Probability„„Monthly Weather Review", 78 (1)

Christ, Miranda, et al. "Undetectable Watermarks for Language Models." *Proceedings of Machine Learning Research*, no. 196, 2024, pp. 1–15, proceedings.mlr.press/v247/christ24a/christ24a.pdf.

Colla, D., Delsanto, M., Agosto, M., Vitiello, B., & Radicioni, D. P. (2022). Semantic coherence markers: The contribution of perplexity metrics. *Artificial Intelligence in Medicine*, *134*, 102393. https://doi.org/10.1016/j.artmed.2022.102393

Das, M., Hussain, Z., & Nawaz, M. (2026). Latency-Aware Benchmarking of Large Language Models for Natural-Language Robot Navigation in ROS 2. *Sensors*, *26*(2), 608. https://doi.org/10.3390/s26020608

Gowal, Sven, et al. "SynthID-Image: Image Watermarking at Internet Scale." *ArXiv.org*, 2025, arxiv.org/abs/2510.09263.

Jafari, M., & Ansari-Pour, N. (2019). Why, When and How to Adjust Your P Values? *Cell Journal (Yakhteh)*, *20*(4), 604–607. https://doi.org/10.22074/cellj.2019.5992

Jiang, Ya, et al. "StealthInk: A Multi-Bit and Stealthy Watermark for Large Language Models." *ArXiv.org*, 2025, arxiv.org/abs/2506.05502.

Jovanović, Nikola, et al. "Watermark Stealing in Large Language Models." *ArXiv.org*, 2024, arxiv.org/abs/2402.19361.

Khachaturov, D., Mullins, R., Shumailov, I., & Dathathri, S. (2025). *Watermarking Needs Input Repetition Masking*. ArXiv.org. https://arxiv.org/abs/2504.12229

Kirchenbauer, John, et al. "A Watermark for Large Language Models." *ArXiv.org*, 6 June 2023, arxiv.org/abs/2301.10226.

Kirchenbauer, John, et al. "On the Reliability of Watermarks for Large Language Models." *ArXiv.org*, 2023, arxiv.org/abs/2306.04634.

Li, J., Galley, M., Brockett, C., Gao, J., & Dolan, B. (2016). *A Diversity-Promoting Objective Function for Neural Conversation Models*. *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. https://doi.org/10.18653/v1/n16-1014

Ren, Jie, et al. "A Robust Semantics-Based Watermark for Large Language Models against Paraphrasing". *Arxiv.org*, 2024, arxiv.org/abs/2311.08721.

Ruibo, Chen, et al. "De-Mark: Watermark Removal in Large Language Models." *Arxiv.org*, 2024, arxiv.org/html/2410.13808v2.

Xiaojun, Xu, et al. "Robust Multi-Bit Text Watermark with LLM-Based Paraphrasers." *Arxiv.org*, 2024, arxiv.org/html/2412.03123v1.

Yoo, KiYoon, et al. "Advancing beyond Identification: Multi-Bit Watermark for Large Language Models." *ArXiv (Cornell University)*, vol. 1, 1 Jan. 2024, pp. 4031–4055, https://doi.org/10.18653/v1/2024.naacl-long.224.